Advanced Segmented, Paged Memory Model Simulator

Lab Report – A9 – Address Translation

Santiago Soto

CS375 – Operating Systems

October 20, 2026

Source: `memory_simulator.cpp` Build: `g++ -std=c++17 -O2 -o memory_simulator memory_simulator.cpp`

Contents

1 Overview **2**
1.1 Implemented features 2
1.2 Translation algorithm 2
1.3 Corrections to the starter code 3

2 Test Configuration **3**

3 Results **3**
3.1 Manual translations 3
3.2 Random address stress test 5
3.3 Batch processing and timeline 7
3.4 FIFO vs LRU 8
3.5 Page size 1000 vs 4000 9
3.6 Two-level page table on a sparse segment 9

4 Short-Answer Questions **10**

5 Long-Answer Questions **11**

6 Challenges and Enhancements **16**

References **16**

1 Overview

The simulator models a virtual memory system that combines segmentation with two-level paging. A logical address is the tuple

$$(\textit{segment},\ \textit{directory index},\ \textit{page},\ \textit{offset},\ \textit{access})$$

and translates to a physical address

$$\textit{physical} = \textit{frame} \times \textit{page_size} + \textit{offset}.$$

Each segment has a *base address* (the location of its page directory), a *limit* in pages and a protection mode (read-only or read-write). The segment's page directory (`DirectoryTable`) points to second-level page tables (`PageTable`). These are created only when a region is first used. Every page-table entry holds a frame number, a present bit, its own protection and a last-access time. A fixed-size `PhysicalMemory` pool tracks which page owns each frame and supports FIFO and LRU replacement.

1.1 Implemented features

Requirement	Implementation
Protection enforcement	segment *and* page RO/RW checks; access type R/W in all inputs
Page replacement	FIFO (load-order queue) and LRU (per-frame last use), switchable at runtime
Multi-level paging	`DirectoryTable` → lazily allocated `PageTable`s
Dynamic configuration	prompts with defaults/validation, config file, *Reconfigure* menu item
Stress testing	N random addresses with a configurable valid ratio → `results.txt`
Utilization tracking	frames used / total, reported after every batch
Batch processing	addresses from a file, per-line results and summary
Simulated latency	1–5 units per translation, +10 for a serviced page fault, history across runs
Error logging	detailed reasons (e.g. "Invalid segment 5, max 2") in `errors.log`
Visualization	memory map (segments, directories, pages, frame map) and Gantt timeline
Experiments	FIFO vs LRU on three workloads; page size 1000 vs 4000

1.2 Translation algorithm

1. Look up the segment. If it does not exist ⇒ *segmentation fault*.
2. Check the directory index, the page number (against the segment limit) and the offset (against the page size) ⇒ *directory fault*, *limit violation*, *offset fault*.
3. Write to a read-only segment ⇒ *protection violation*.
4. If the directory entry is empty, allocate a page table for it.
5. Write to a read-only page ⇒ *protection violation*. If the present bit is 0 ⇒ *page fault*: use a free frame, or else evict the FIFO/LRU victim and clear the victim's present bit.
6. Compute the physical address and update LRU/FIFO bookkeeping and metrics.

1.3 Corrections to the starter code

- The starter computed `base + frame * page_size + offset`. Frames already have absolute positions, so adding the segment base relocated twice and made segments overlap (segment 0 frame 12 = segment 1 frame 2). The simulator uses `frame * page_size + offset`. The base locates the page directory.
- The starter's fault handler called `setFrame(page, frame, accessType)`. A write to a non-present read-only page therefore *upgraded* that page to read-write. Protection is now a fixed attribute of the page and is checked before any fault is serviced.
- The starter gave initial pages `rand() % 100` as frame numbers, ignoring the frame pool (pages could alias or point past physical memory). Initial pages now claim real free frames with recorded owners, so eviction can update the right page table.

2 Test Configuration

All screenshots use `sample_config.txt` with seed 42, so the runs are reproducible:

```
frames 7
page_size 1000
pages_per_dir 4
policy LRU
0  0x1000  6   RO   2:RO 5:RO -:RO -:RO             # code
1  0x2000  8   RW   0:RW -:RW 6:RW -:RO 3:RW -:RW    # data
2  0x3000  12  RW   1:RW -:RW                        # stack
```

This gives 26 virtual pages over 7 frames (3.71× over-committed). Six pages are resident at start, and one frame (4) is free. Segment 0 directory 1 and segment 2 directories 1–2 start unallocated.

```
root@Saint-James:~/Module-5-Assignment-1-Address-Translation/submission# cd .. && make clean && make && cd submission
rm -rf build
rm -f report/*.aux report/*.log report/*.out report/*.toc report/*.pdf
g++ -std=c++17 -Wall -Wextra -O2 -o build/memory_simulator submission/memory_simulator.cpp
root@Saint-James:~/Module-5-Assignment-1-Address-Translation/submission#
```

Figure 1: Building the simulator with `-Wall -Wextra` (no warnings).

3 Results

3.1 Manual translations

Five manual translations mixing valid and invalid addresses:

#	Logical address	Outcome	Latency
1	(1,0,2,250,R)	hit in frame 6 → physical 6250	3
2	(0,0,1,999,W)	protection violation (segment 0 is read-only)	3
3	(2,1,2,40,W)	directory allocated, page fault → free frame 4 → 4040	12
4	(5,0,0,0,R)	segmentation fault: invalid segment 5, max 2	5
5	(1,1,3,100,R)	page fault, memory full, LRU evicts S1:D0:P0 from frame 0 → 100	12

```
  MANUAL ADDRESS TRANSLATION
============================================================
Enter: seg dir page offset access   (access: R/W or 0=read, 1=write)
Example: 1 0 2 250 R     Commands: m = memory map, q or -1 = back to menu

addr> 1 0 2 250 R

  #1 (1,0,2,250,R)
     - Segment 1: page directory at 0x2000, limit 8 pages, RW
     - Directory 0: -> page table with 4 entries
     - Page 2: present in frame 6 (RW)
     - Physical address = frame 6 x 1000 + offset 250 = 6250
     => HIT  Physical address: 6250   latency 3

addr> 0 0 1 999 W

  #2 (0,0,1,999,W)
     - Segment 0: page directory at 0x1000, limit 6 pages, RO
     => PROTECTION VIOLATION: Cannot write to read-only segment 0   latency 3

addr> 2 1 2 40 W

  #3 (2,1,2,40,W)
     - Segment 2: page directory at 0x3000, limit 12 pages, RW
     - Directory 1: entry was empty -> allocated a new 4-entry page table
     - Page 2: present bit = 0 -> PAGE FAULT
     - Loaded page into free frame 4
     - Physical address = frame 4 x 1000 + offset 40 = 4040
     => PAGE FAULT serviced  Physical address: 4040   latency 12

addr> 5 0 0 0 R

  #4 (5,0,0,0,R)
     => SEGMENTATION FAULT: Invalid segment 5, max 2   latency 5

addr> 1 1 3 100 R

  #5 (1,1,3,100,R)
     - Segment 1: page directory at 0x2000, limit 8 pages, RW
     - Directory 1: -> page table with 4 entries
     - Page 3: present bit = 0 -> PAGE FAULT
     - Memory full: LRU evicted S1:D0:P0 from frame 0, loaded page there
     - Physical address = frame 0 x 1000 + offset 100 = 100
     => PAGE FAULT serviced  Physical address: 100   latency 12

addr> q

---------- Manual session summary ----------
Translations requested : 5
Successful             : 3  (60.0%)
Faults / errors        : 2  (40.0%)
    Segmentation faults   : 1
    Directory faults      : 0
    Limit violations      : 0
    Offset faults         : 0
    Protection violations : 1
Page faults (serviced) : 2  (rate 40.0% of requests, 66.7% of valid references)
Page replacements      : 1  (LRU)
Page tables allocated  : 1 on demand
Memory utilization     : 7/7 frames (100.0%)
Average latency        : 7.00 units  (total 35)
root@Saint-James:~/Module-5-Assignment-1-Address-Translation/submission#
```

Figure 2: Sample run: five manual address translations (valid and invalid).

```
  MEMORY MAP
============================================================
Policy LRU | Frames 7 | Page size 1000 B | Pages/directory 4 | Clock 35

Segment table
  Seg  Dir base   Limit  Prot  Directory entries (allocated)
  0    0x1000     6      RO    2 (1)
  1    0x2000     8      RW    2 (2)
  2    0x3000     12     RW    3 (2)

Segment 0 [RO]
  Dir 0: page table, 4 entries
    Page 0 (vpn  0): frame 2  present=1 RO  last=0
    Page 1 (vpn  1): frame 5  present=1 RO  last=0
    Page 2 (vpn  2): --       present=0 RO
    Page 3 (vpn  3): --       present=0 RO
  Dir 1: (not allocated - no page table)

Segment 1 [RW]
  Dir 0: page table, 4 entries
    Page 0 (vpn  0): --       present=0 RW
    Page 1 (vpn  1): --       present=0 RW
    Page 2 (vpn  2): frame 6  present=1 RW  last=1
    Page 3 (vpn  3): --       present=0 RO
  Dir 1: page table, 4 entries
    Page 0 (vpn  4): frame 3  present=1 RW  last=0
    Page 1 (vpn  5): --       present=0 RW
    Page 2 (vpn  6): --       present=0 RW
    Page 3 (vpn  7): frame 0  present=1 RW  last=5

Segment 2 [RW]
  Dir 0: page table, 4 entries
    Page 0 (vpn  0): frame 1  present=1 RW  last=0
    Page 1 (vpn  1): --       present=0 RW
    Page 2 (vpn  2): --       present=0 RW
    Page 3 (vpn  3): --       present=0 RW
  Dir 1: page table, 4 entries
    Page 0 (vpn  4): --       present=0 RW
    Page 1 (vpn  5): --       present=0 RW
    Page 2 (vpn  6): frame 4  present=1 RW  last=3
    Page 3 (vpn  7): --       present=0 RW
  Dir 2: (not allocated - no page table)

Physical frames
  F00 [S1:D1:P3   ]  F01 [S2:D0:P0   ]  F02 [S0:D0:P0   ]  F03 [S1:D1:P0   ]
  F04 [S2:D1:P2   ]  F05 [S0:D0:P1   ]  F06 [S1:D0:P2   ]

Physical memory utilization: 7/7 frames (100.0%)
Page-table entries: two-level = 7 directory + 20 page-table = 27  |  single-level would need 26
root@Saint-James:~/Module-5-Assignment-1-Address-Translation/submission#
```

Figure 3: Memory map after the five translations: segment table, directories, page tables and frame ownership.

3.2 Random address stress test

100 random addresses with a valid ratio of 0.70 (72 in range, 28 out of range):

Metric	Value
Successful translations	58 (58%)
Segmentation / directory / limit / offset faults	9 / 7 / 7 / 5
Protection violations	14
Page faults serviced	41 (41% of requests, 70.7% of valid references)
Page replacements (LRU)	40
Memory utilization	7/7 frames (100%)
Average latency	7.18 units

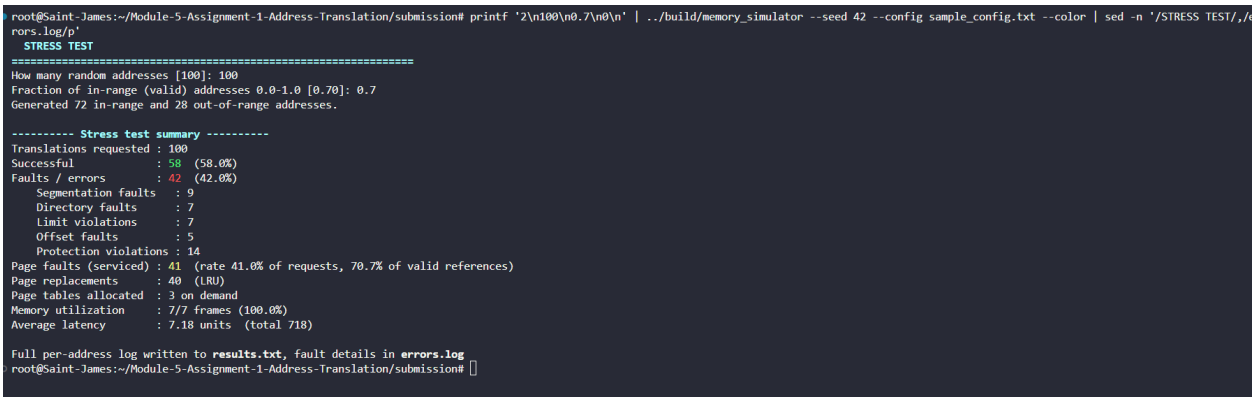


Figure 4: Stress test summary printed on screen.

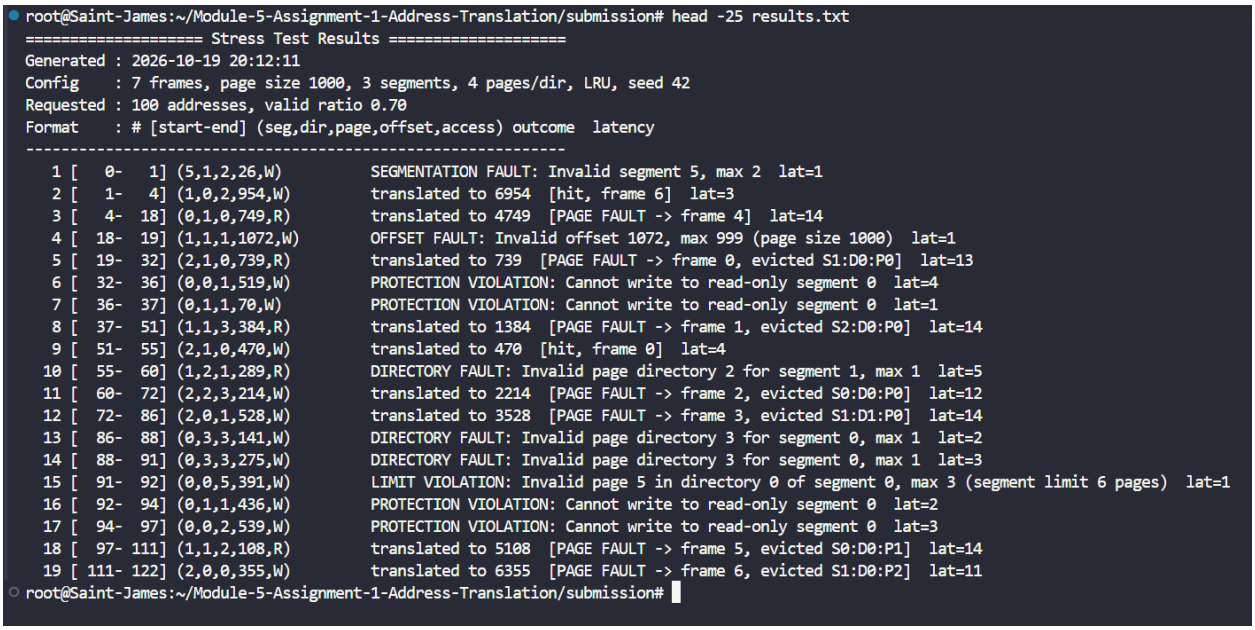


Figure 5: `results.txt`: header and first per-address entries with timeline `[start-end]`.

```
root@Saint-James:~/Module-5-Assignment-1-Address-Translation/submission# tail -17 results.txt
------------------------------------------------------------
Generated 72 in-range and 28 out-of-range addresses

---------- Stress test summary ----------
Translations requested : 100
Successful             : 58  (58.0%)
Faults / errors        : 42  (42.0%)
    Segmentation faults   : 9
    Directory faults      : 7
    Limit violations      : 7
    Offset faults         : 5
    Protection violations : 14
Page faults (serviced) : 41  (rate 41.0% of requests, 70.7% of valid references)
Page replacements      : 40  (LRU)
Page tables allocated  : 3 on demand
Memory utilization     : 7/7 frames (100.0%)
Average latency        : 7.18 units  (total 718)
root@Saint-James:~/Module-5-Assignment-1-Address-Translation/submission#
```

Figure 6: `results.txt`: summary section.

```
root@Saint-James:~/Module-5-Assignment-1-Address-Translation/submission# head -25 errors.log
=== Error / fault log - 2026-10-19 20:12:11 - file sample_config.txt, 7 frames, page size 1000, LRU, seed 42 ===
[t=0] #1 (5,1,2,26,W)  SEGMENTATION FAULT: Invalid segment 5, max 2
[t=4] #3 (0,1,0,749,R)  PAGE FAULT (serviced): loaded into frame 4, frame was free
[t=18] #4 (1,1,1,1072,W)  OFFSET FAULT: Invalid offset 1072, max 999 (page size 1000)
[t=19] #5 (2,1,0,739,R)  PAGE FAULT (serviced): loaded into frame 0, LRU evicted S1:D0:P0
[t=32] #6 (0,0,1,519,W)  PROTECTION VIOLATION: Cannot write to read-only segment 0
[t=36] #7 (0,1,1,70,W)  PROTECTION VIOLATION: Cannot write to read-only segment 0
[t=37] #8 (1,1,3,384,R)  PAGE FAULT (serviced): loaded into frame 1, LRU evicted S2:D0:P0
[t=55] #10 (1,2,1,289,R)  DIRECTORY FAULT: Invalid page directory 2 for segment 1, max 1
[t=60] #11 (2,2,3,214,W)  PAGE FAULT (serviced): loaded into frame 2, LRU evicted S0:D0:P0
[t=72] #12 (2,0,1,528,W)  PAGE FAULT (serviced): loaded into frame 3, LRU evicted S1:D1:P0
[t=86] #13 (0,3,3,141,W)  DIRECTORY FAULT: Invalid page directory 3 for segment 0, max 1
[t=88] #14 (0,3,3,275,W)  DIRECTORY FAULT: Invalid page directory 3 for segment 0, max 1
[t=91] #15 (0,0,5,391,W)  LIMIT VIOLATION: Invalid page 5 in directory 0 of segment 0, max 3 (segment limit 6 pages)
[t=92] #16 (0,1,1,436,W)  PROTECTION VIOLATION: Cannot write to read-only segment 0
[t=94] #17 (0,0,2,539,W)  PROTECTION VIOLATION: Cannot write to read-only segment 0
[t=97] #18 (1,1,2,108,R)  PAGE FAULT (serviced): loaded into frame 5, LRU evicted S0:D0:P1
[t=111] #19 (2,0,0,355,W)  PAGE FAULT (serviced): loaded into frame 6, LRU evicted S1:D0:P2
[t=122] #20 (2,1,4,407,R)  LIMIT VIOLATION: Invalid page 4 in directory 1 of segment 2, max 3 (segment limit 12 pages)
[t=126] #21 (0,0,4,800,W)  LIMIT VIOLATION: Invalid page 4 in directory 0 of segment 0, max 3 (segment limit 6 pages)
[t=131] #23 (0,1,1,487,W)  PROTECTION VIOLATION: Cannot write to read-only segment 0
[t=134] #24 (1,1,0,662,W)  PAGE FAULT (serviced): loaded into frame 4, LRU evicted S0:D1:P0
[t=145] #25 (2,1,4,846,W)  LIMIT VIOLATION: Invalid page 4 in directory 1 of segment 2, max 3 (segment limit 12 pages)
[t=148] #26 (0,1,1,931,R)  PAGE FAULT (serviced): loaded into frame 1, LRU evicted S1:D1:P3
[t=163] #27 (1,0,0,1019,R)  OFFSET FAULT: Invalid offset 1019, max 999 (page size 1000)
root@Saint-James:~/Module-5-Assignment-1-Address-Translation/submission#
```

Figure 7: `errors.log`: detailed fault reasons.

3.3 Batch processing and timeline

`batch_addresses.txt` holds 13 addresses and one malformed line, which is reported and skipped. Result: 7 successful, 6 errors (one of each fault type plus two protection violations), 3 page faults, 2 replacements, average latency 5.15 units.

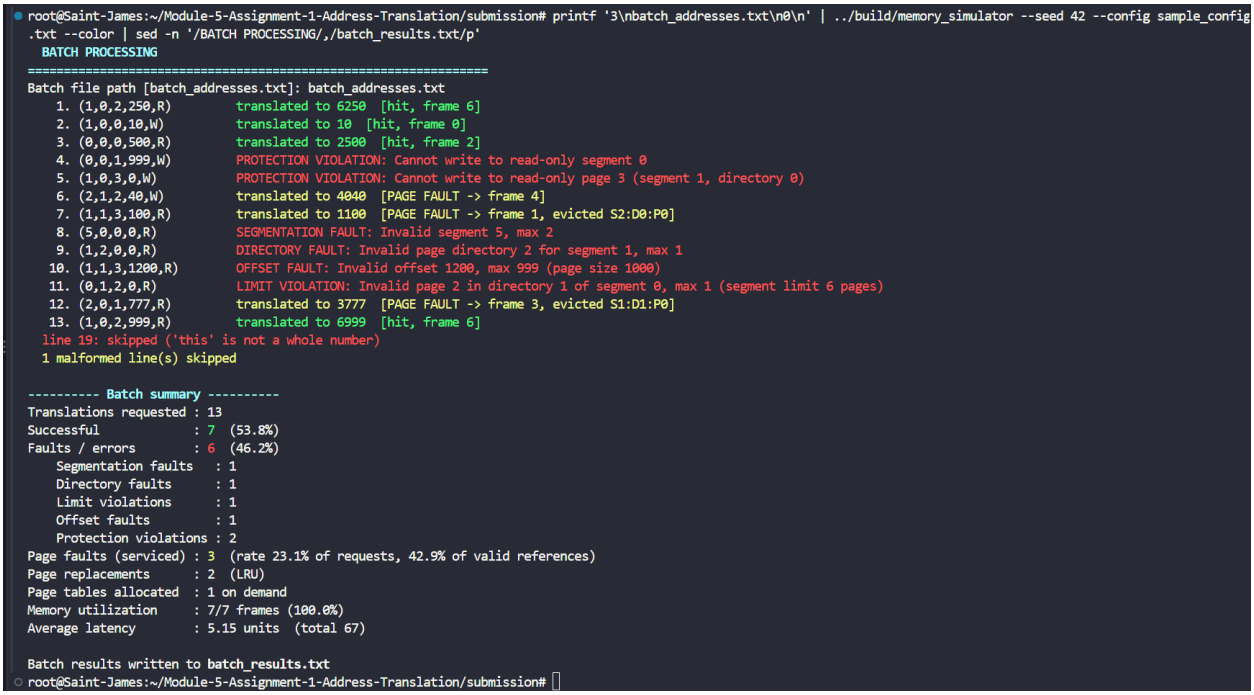


Figure 8: Batch processing with per-line results and summary.

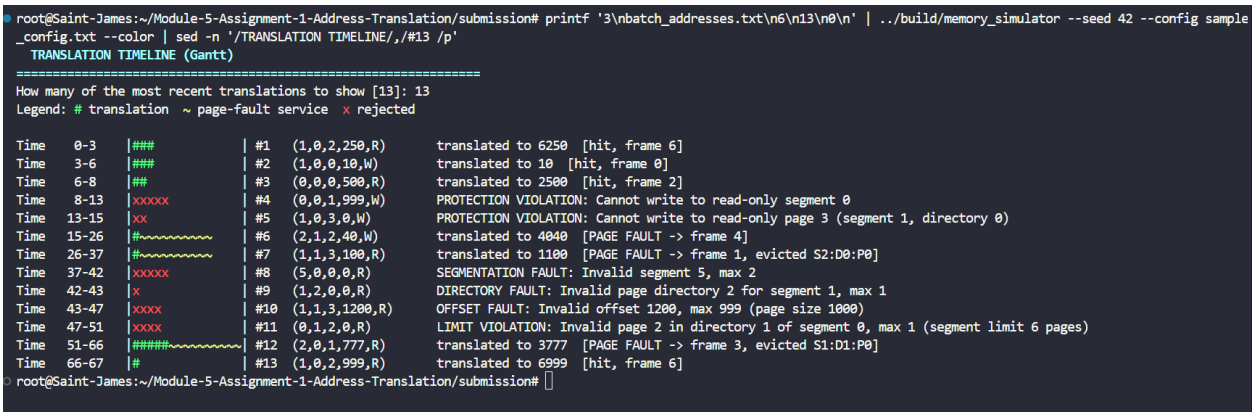


Figure 9: Gantt-like timeline: `#` translation time, `~` page-fault service, `x` rejected request.

3.4 FIFO vs LRU

Each workload (1000 valid reads) is replayed from the same starting state with each policy:

Workload	FIFO faults	LRU faults	Better
Uniform random	733 (73.3%)	732 (73.2%)	≈ tie
Locality 80/20 (hot set 5 pages)	306 (30.6%)	216 (21.6%)	LRU
Loop over 8 pages (7 frames)	502 (50.2%)	998 (99.8%)	FIFO

```
root@Saint-James:~/Module-5-Assignment-1-Address-Translation/submission# printf '7\n1000\n0\n' | ../build/memory_simulator --seed 42 --config sample_config.txt --color |
sed -n '/FIFO vs LRU COMPARISON/,/unchanged/p'
  FIFO vs LRU COMPARISON
============================================================
Each workload is replayed from the CURRENT memory state on a copy that uses FIFO
and a copy that uses LRU. All references are valid reads, so only paging differs.
References per workload [1000]: 1000

Workload              FIFO faults        LRU faults         Fewer faults
------------------------------------------------------------------------
Uniform random        733 (73.3%)        732 (73.2%)        LRU
Locality 80/20        306 (30.6%)        216 (21.6%)        LRU
Loop over 8 pages     502 (50.2%)        998 (99.8%)        FIFO

7 frames, 26 virtual pages, hot set = 5 pages. (The live simulator state is unchanged.)
root@Saint-James:~/Module-5-Assignment-1-Address-Translation/submission#
```

Figure 10: FIFO vs LRU comparison.

3.5 Page size 1000 vs 4000

The same program (segments of 5829, 7359 and 11646 bytes), the same 16000 bytes of physical memory and the same 2000-reference byte trace, run from a cold start:

	1000 B pages	4000 B pages
Physical frames	16	4
Virtual pages (single-level PTEs)	26	7
Page-table memory (single-level)	104 B	28 B
Internal fragmentation	1166 B	3166 B
Page faults	169 (8.4%)	195 (9.8%)
Average latency	3.88	4.01

```
root@Saint-James:~/Module-5-Assignment-1-Address-Translation/submission# printf '8\n1000\n4000\n2000\n16000\n0\n' | ../build/memory_simulator --seed 42 --config sample_co
nfig.txt --color | sed -n '/PAGE-SIZE ANALYSIS/,/unchanged/p'
  PAGE-SIZE ANALYSIS
============================================================
Simulates the same program (same segment sizes in bytes, same physical memory in
bytes, same byte-level reference trace) under two page sizes, from a cold start.
First page size [1000]: 1000
Second page size [4000]: 4000
Number of references [2000]: 2000
Physical memory in bytes (same for both) [16000]: 16000

Segment sizes (bytes):  S0=5829  S1=7359  S2=11646
Physical memory: 16000 bytes, policy LRU, pages/dir 4

                                    page size 1000    page size 4000
  ------------------------------------------------------------------------
  Physical frames                   16                4
  Virtual pages (= single-level PTEs)  26             7
  Two-level entries actually allocated  33            10
  Page-table memory (single-level)  104 B             28 B
  Internal fragmentation            1166 B            3166 B
  Page faults                       169 (8.4%)        195 (9.8%)
  Replacements                      153               191
  Average latency (units)           3.88              4.01

(Page-fault service adds 10 units. The live simulator state is unchanged.)
root@Saint-James:~/Module-5-Assignment-1-Address-Translation/submission#
```

Figure 11: Page-size analysis.

3.6 Two-level page table on a sparse segment

`sparse_config.txt` defines a 1024-page segment with 32 pages per directory entry. Touching the heap start and the stack top allocates only 2 page tables. Two-level total: $33 + 80 = 113$ entries. A single-level table would need 1040 entries, so the two-level table uses 89% less.

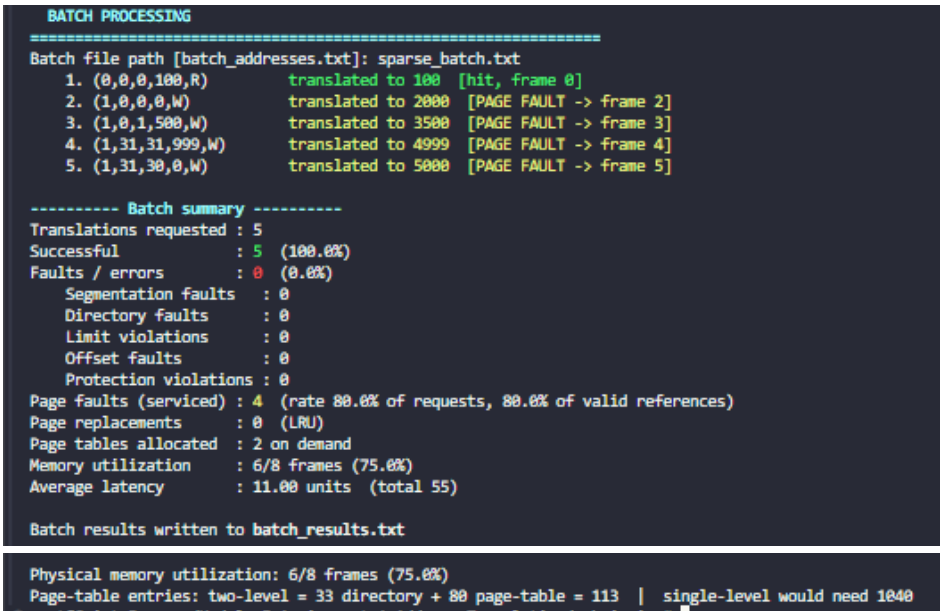


Figure 12: Sparse segment: batch run (top) and the page-table entry count, two-level vs. single-level (bottom).

4 Short-Answer Questions

1. What is the role of the `present` bit in the page table, and how does it trigger a page fault?

The present bit records whether a page is resident in a physical frame. When a legal access finds it set to 0, `getFrameNumber` returns *not present*. The handler then raises a page fault, loads the page into a free or victim frame, sets the bit to 1 and completes the access.

2. How does a protection violation differ from a segmentation fault?

A segmentation fault means the address itself is invalid: no such segment, or the address is outside its limit. A protection violation means the address is valid but the access type is forbidden, for example writing to a read-only segment or page.

3. Why is a two-level page table useful for large address spaces?

Only the small directory must be fully allocated. Second-level tables are created only for regions that are actually used, so a sparse address space needs far fewer entries. In the sparse test this was 113 entries instead of 1040.

4. Explain the purpose of the `base_address` in the segment table.

The base address tells the translator where the segment's page directory starts in physical memory, like a per-segment page-table base register. Together with the limit, it defines the segment. In pure segmentation, the base is instead the start of the segment, and the offset is added to it.

5. How does changing the page size affect the number of page table entries and offset range?

A larger page size means fewer entries ($\lceil \text{size}/\text{page size} \rceil$) and a wider offset range ($0 \ldots \text{page size} - 1$, i.e. more offset bits). A smaller page size means more entries and a narrower offset range. In the test, 26 entries at 1000 B dropped to 7 entries at 4000 B.

6. What is the main advantage of FIFO over LRU for page replacement?

FIFO is simpler and cheaper. It only updates a queue when a page is loaded. LRU must record every reference and search for (or maintain an order of) the least recently used frame.

7. Why might a read-only segment be used in a real OS?

Read-only segments protect code and constant data from accidental or malicious modification, such as bugs or code injection. They also let many processes safely share one physical copy of a program or library.

5 Long-Answer Questions

8. Describe how the segment table and page table work together to translate a logical address. Use an example from your simulator with specific values.

Translation is a chain of lookups. Each level narrows the search and adds a check. The segment table is consulted first. The segment number selects an entry that holds the base address of the segment's page directory, its limit in pages and its protection. The directory index then selects a directory entry, which points to a second-level page table. Inside that table, the page number selects a page-table entry with a frame number, a present bit and page-level protection. Finally, the offset is appended within the frame: $\textit{physical} = \textit{frame} \times \textit{page size} + \textit{offset}$.

Take translation #1 from Figure 2, the logical address (1, 0, 2, 250, Read). Segment 1 exists (valid range 0–2). Its entry says the page directory is at `0x2000`, the limit is 8 pages and the protection is read-write. Directory index 0 is inside the two-entry directory ($\lceil 8/4 \rceil = 2$) and points to an allocated 4-entry page table. Page 2 is below 4 and corresponds to virtual page $0 \times 4 + 2 = 2$, within the limit. The offset 250 is below the page size of 1000. A read is allowed. The entry for page 2 has present = 1 and frame 6, so the result is $6 \times 1000 + 250 = \mathbf{6250}$, with a simulated latency of 3 units.

Other examples show what happens when a lookup cannot finish normally. For (2, 1, 2, 40, Write) the segment-level checks pass, but directory entry 1 of segment 2 is empty. The simulator first allocates a page table for that region. The page's present bit is 0, so a page fault is raised. Frame 4 is still free, so the page is loaded there and the address becomes 4040. The latency is $2 + 10$ units, because the fault-service penalty is added. For (1, 1, 3, 100, Read), virtual page $1 \times 4 + 3 = 7$ is not resident and all 7 frames are in use. LRU chooses the frame with the oldest last use, frame 0 (owned by S1:D0:P0). It clears that page's present bit, loads page 7 into frame 0 and returns physical address 100. Each level validates its part of the address first: the segment table checks existence, limit and segment rights, and the page table checks presence and page rights. This is why every error can be reported precisely, for example "Invalid segment 5, max 2".

9. Explain the trade-offs of FIFO vs. LRU page replacement in the context of your simulator. Which performs better for random workloads, and why?

In the simulator, FIFO uses a queue of frames in load order. A page fault evicts the front of the queue, and the reloaded frame moves to the back. LRU stores a last-use time per frame. This value is updated on *every* successful reference, and a fault scans the frames for the minimum. So FIFO's cost is paid only on faults, while LRU's bookkeeping is paid on every access, plus an $O(\text{frames})$ search per fault. Real hardware can rarely afford exact LRU, which is why approximations such as the clock algorithm exist. FIFO can also show Bélády's anomaly, where more frames cause more faults. LRU is a stack algorithm and cannot.

The comparison experiment (Figure 10) replays the same reference strings from the same memory state under both policies. With a *uniform random* workload, FIFO had 733 faults and LRU

732 out of 1000. That is effectively a tie, and it is what theory predicts: when every page is equally likely next, recency carries no information. Any policy that keeps some 7 of the 26 pages has about the same hit chance ($7/26 \approx 27\%$, i.e. about a 73% fault rate). For random workloads, then, neither policy wins. FIFO is the better engineering choice there because it gives the same result for less work.

The picture changes once the workload has structure. With 80/20 locality (80% of references go to a 5-page hot set), LRU had 216 faults against FIFO's 306, about 29% fewer. LRU keeps the hot pages because they are always recently used. FIFO evicts them as soon as they reach the head of the queue, however often they are used. The third workload loops over 8 pages with only 7 frames. This is LRU's worst case: the page it evicts is always the one needed next, so it faulted on 998 of 1000 references. FIFO faulted on 502, because the loop and the queue drift out of phase. The conclusion is that LRU is better for typical programs, which have locality, FIFO is as good for random access, and cyclic access patterns larger than memory defeat LRU.

10. Discuss the challenges of implementing protection checks in your simulator. How do they improve system security, and what are the potential performance costs?

The first challenge was *where* the checks belong. Protection exists at two levels. A segment can be read-only as a whole (the code segment), and individual pages inside a read-write segment can also be read-only (page 3 of segment 1). The simulator checks the segment first and then the page entry. A page cannot be more permissive than its segment, so the config loader forces pages of a read-only segment to read-only.

The second challenge was *ordering*, and the starter code shows why it matters. The starter checked the present bit before protection. On a page fault it then called `setFrame(page, frame, accessType)`, which overwrote the page's protection with the access type. A write to a non-resident read-only page therefore faulted, the page was loaded *as read-write* and the write succeeded. That is a privilege escalation caused by a missing check. In the simulator, protection is a fixed attribute of the page-table entry, independent of residency. It is checked before the fault handler runs, so an illegal write never loads a page or changes memory state. The stress test shows this works: even at a valid ratio of 1.0, all 16 failures were protection violations from random writes to read-only regions.

Protection improves security in several ways. It keeps code from being overwritten, which blocks a whole class of code-injection attacks. It isolates a process's constant data. It also makes sharing safe: many processes can map one physical copy of a library only if none of them can write it. Every violation is logged with a reason in `errors.log`, which in a real OS corresponds to delivering a signal such as SIGSEGV and auditing the event.

The cost per translation is small: two comparisons, which in hardware are done in parallel by the MMU, with the permission bits cached in the TLB beside the frame number. The larger costs are indirect. Permission bits make TLB entries larger. Changing a page's protection requires a TLB shootdown on multiprocessor systems. Each violation also traps into the kernel, which costs a context switch. In the simulator, a rejected access still pays its 1–5 latency units, but it never pays the 10-unit fault penalty, so failing early is also the cheaper path.

11. Analyze the impact of page size on memory efficiency and translation overhead. Use your simulator to illustrate with two different page sizes (e.g., 1000 vs. 4000).

Page size is a trade-off between *page-table overhead* and *internal fragmentation*, and it also affects how well memory can follow a program's working set. The page-size analysis (Figure 11) runs the same program under both sizes: three segments of 5829, 7359 and 11646 bytes, 16000 bytes of physical memory and the same 2000-reference byte-level trace with 80/20 locality.

Table overhead. At 1000 bytes per page, the program needs 26 pages, so a single-level design needs 26 entries (104 bytes). At 4000 bytes it needs only 7 entries (28 bytes), a 3.7× reduction. The offset grows from 0–999 to 0–3999. Fewer, larger pages also mean that each TLB entry covers four times as much memory, so the TLB misses less often and fewer translations need a page walk.

Internal fragmentation. Each segment's last page is only partly used. With 1000-byte pages the waste totals 1166 bytes. With 4000-byte pages it grows to 3166 bytes, almost 20% of the 16000-byte memory. Large pages waste memory in the last page of every segment.

Fault behaviour. The same 16000 bytes provide 16 small frames or only 4 large ones. With 4000-byte pages, each fault brings in more data, which helps sequential access (prefetching). But memory can hold only 4 distinct regions, so a hot set spread over three segments competes for very few slots. Small pages let memory hold exactly the hot parts. The result was 169 faults (8.4%) with 1000-byte pages against 195 (9.8%) with 4000-byte pages, and average latency rose from 3.88 to 4.01 units. In a real system each large-page fault also transfers four times more data from disk, which the simulator's flat 10-unit penalty understates.

Larger pages therefore lower translation overhead (smaller tables, better TLB reach), and smaller pages use memory more efficiently (less fragmentation, finer working sets). Real systems use a moderate default (4 KB) and offer optional huge pages (2 MB/1 GB) for large, dense regions such as databases.

12. Explain how multi-level page tables reduce memory usage compared to single-level tables. Provide a scenario where your two-level implementation is beneficial.

A single-level page table needs one entry for *every* virtual page in the address space, whether or not the page is ever used. The table is indexed directly by page number, so it must be contiguous and fully allocated. For a 32-bit space with 4 KB pages that is 2^{20} entries (4 MB) per process. Most processes, however, use only a few compact regions, such as code and heap at the bottom and the stack at the top, with a huge unused gap between them.

A two-level table splits the page number into a directory index and a page index. The directory is small and always present. A second-level table is allocated only when some page in its range is used, and an empty directory entry stands for a whole unused range of pages at the cost of one entry. Memory use therefore grows with the parts of the address space actually used, not with its total size. The cost is one extra memory access per translation on a TLB miss.

The simulator implements this in `DirectoryTable`. Each directory entry is a `std::optional<PageTable>` that stays empty until the first access to its range. That access appears in the trace as "entry was empty → allocated a new page table". The memory map prints unallocated entries as "(not allocated – no page table)", and the metrics compare the two-level footprint with the single-level equivalent.

A scenario where this helps is `sparse_config.txt`, which models a process with a 16-page code segment and a 1024-page (about 1 MB) data segment for heap and stack, with 32 pages per directory entry. The program touches only the start of the heap (directory 0) and the top of the stack (directory 31). Only those two page tables are created. The total is 33 directory entries +80 page-table entries = 113, compared with 1040 for a single-level table. That is 452 bytes instead of 4160 bytes, 89% less (Figure 12). The saving grows with sparseness: if the stack later grows downward, only the directories it reaches are allocated.

For small, dense segments, two levels can cost slightly *more*. In the sample configuration, the two-level total was 27–33 entries against 26 for a single-level table, because the directory entries are overhead. Multi-level tables pay off for large, sparse address spaces, which is the common case in real systems.

13. Describe how your simulator handles dynamic memory configuration. Discuss

challenges in ensuring valid configurations (e.g., preventing over-allocation).

The configuration can be set three ways. (1) At start-up, interactive prompts ask for the number of frames, page size, number of segments, pages per directory entry and replacement policy. Each prompt shows a default and re-asks on invalid input, for example "Please enter a whole number from 1 to 4096". Segments are then created randomly: limits of 4–12 pages, about a quarter read-only, some directory entries left empty and about 40% of pages resident in random free frames. (2) A configuration file (`-config` or the start-up prompt) gives exact control. Each segment line lists its ID, base, limit, protection and per-page tokens such as `6:RW` or `-:RO`. (3) The *Reconfigure* menu item rebuilds memory at runtime without restarting the program. The policy can also be switched at any time.

Validation happens at several layers. Each value is range-checked: frames 1–4096, page size up to 1 000 000, at most 32 segments, limits 1–4096 pages. Derived quantities are checked too: frames × page size must fit in the address range, otherwise physical addresses would overflow. Structural rules are enforced: segment IDs must be sequential, because the segment number indexes the table directly. A segment cannot list more pages than its limit. Page directories may not overlap in memory, a read-only segment cannot contain writable pages, and frame numbers must exist. The file loader collects *all* errors with line numbers and rejects the whole configuration, so a half-loaded state can never exist.

Over-allocation was the subtlest challenge, because two different conditions both look like "too many pages". Having more *virtual* pages than frames is legal; that is the purpose of virtual memory. The sample configuration has 26 pages over 7 frames, and the simulator reports this as a note ("3.71× over-committed, demand paging will be used"), not an error. What must never happen is more *resident* pages than frames, or two pages sharing one frame. The starter's random `rand() % 100` frame numbers broke this invariant. The simulator prevents it in three ways: frames are only taken through `claimFrame`, which refuses frames that are in use or out of range; a duplicate frame in a config file is rejected with the name of the page that already owns it; and `validateConfiguration()` checks after start-up that each resident page maps to a distinct, valid frame whose owner record points back to that page.

14. Evaluate the effectiveness of your stress testing approach. How does varying the valid/invalid address ratio reveal system limitations?

The stress test generates N addresses (default 100). With probability r (the valid ratio) an address is *in range*: a random existing segment, a virtual page below the limit, split into directory and page, and a legal offset. Otherwise one field is deliberately broken (a non-existent segment, an out-of-range directory, a page past the limit or an offset past the page size), so every error path is exercised. Access type is random, so in-range addresses can still be protection violations. Every result goes to `results.txt` with its time slot, and every fault reason goes to `errors.log`. With a fixed seed, runs can be reproduced exactly, which allowed the following sweep (seed 42, sample configuration, 100 addresses):

Valid ratio	Successful	Errors	Page faults	Avg latency
0.0	0	100	0	3.09
0.3	22	78	19	5.06
0.5	34	66	29	6.01
0.7	58	42	41	7.18
0.9	71	29	56	8.56
1.0	84	16	58	8.89

The sweep shows several things. At $r = 0$, every request is rejected during validation, before any memory state changes. Latency stays at the bare 1–5 unit range (mean 3.09), which confirms that invalid addresses are cheap and cannot cause page faults or evictions. As r rises, latency increases almost in step with the page faults, because each fault adds 10 units. The real bottleneck of the system is therefore the 7-frame memory, not the checking logic. At $r = 1.0$, 58 of 84 successful references faulted (69%). This is consistent with uniform random access to 26 pages through 7 frames and shows that the configuration thrashes once the working set exceeds memory. Even at $r = 1.0$, 16 requests fail, all of them protection violations. This shows that "in range" is not the same as "allowed", and that the protection logic still works under load.

The approach has limits. Uniformly random addresses have no locality, so they show the worst case for paging and cannot tell FIFO and LRU apart. The dedicated locality and loop workloads in the FIFO/LRU experiment address this. Also, only one field is broken per invalid address, so combined errors test only the first check that fails. Better coverage would replay real program traces, mix in a skewed (Zipf) distribution, and sweep the frame count to find the point where the fault rate drops sharply.

15. Propose a real-world application (e.g., embedded system, cloud server) for your simulator. How would you extend it to handle specific requirements (e.g., real-time constraints, multi-threading)?

One application is a *capacity-planning tool for a multi-tenant cloud server*. A hosting provider must decide how much RAM to give each virtual machine or container, which page size to use, and how far memory can be over-committed before tenants start thrashing. The simulator already models the key mechanisms: protection between regions, demand paging, replacement, utilization and latency metrics. It also reads configurations from files, so an operator could load real memory layouts and replay recorded address traces, then compare policies and page sizes before changing production settings.

Several extensions would be needed for this use.

- **Multiple address spaces.** Today there is one segment table. A server needs one per tenant (process), sharing a single `PhysicalMemory`. Frames would gain an owner process ID, and a TLB with address-space identifiers (ASIDs) would avoid flushing on every context switch. Shared read-only segments (libraries) would map the same frames into several tables, using the existing protection checks.
- **Multi-threading.** Tenants run concurrently, so translation must be thread-safe. The simple approach is a mutex around the frame pool and per-address-space locks on page tables. A better approach is a lock-free read path for TLB/page-table hits, with locks only on page faults and evictions, plus TLB-shootdown simulation when a page is evicted.
- **Better replacement and quotas.** Add the clock (second-chance) algorithm, dirty bits with write-back cost, per-tenant frame quotas and working-set tracking, so that one tenant cannot evict everyone else's pages.
- **Realistic costs.** Replace the flat penalty with a model of DRAM, SSD and network-storage latency, and report tail latency (99th percentile), not only the mean.

The same design could serve an *embedded real-time* controller, such as an automotive ECU, with different extensions. Real-time code cannot tolerate unbounded page-fault delays, so the extension would add *page locking* (pinned frames that are never chosen as victims) for time-critical tasks. The simulator's latency model would also be used to prove a worst-case translation time: each task's deadline would be checked against the timeline, and the run would be flagged if any translation

exceeded its budget. Memory protection would isolate safety-critical tasks from less trusted ones, which is the role of the MPU on microcontrollers.

6 Challenges and Enhancements

Challenges.

- **Eviction needs a reverse mapping.** Replacement picks a *frame*, but it must clear the present bit of the *page* that owns it. Each frame therefore records its owner (segment, directory, page), and a consistency check confirms that owners and page tables agree.
- **Fixing the starter's protection hole and double relocation** (Section 1.3) required separating protection from residency and changing what the base address means.
- **Fair policy comparison.** Comparing FIFO and LRU on different random runs proves nothing. The comparison copies the whole simulator state and replays an identical reference string under each policy.
- **Reproducibility for the report.** All randomness comes from one seeded `std::mt19937`, so `-seed 42` reproduces every number above.

Enhancements beyond the requirements. Step-by-step translation traces, colored output, input that accepts commas and parentheses, input echo for piped sessions, latency history across runs (`latency_history.csv`), the FIFO vs LRU and page-size experiments, sparse-segment demonstration, and a configuration file format with line-numbered errors.

References

[1] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, Arpaci-Dusseau Books, 2018. Chapters 16 (Segmentation), 18–20 (Paging, TLBs, Smaller Tables), 21–22 (Swapping: Mechanisms and Policies). `https://pages.cs.wisc.edu/~remzi/OSTEP/`

[2] A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed., Wiley, 2018. Chapters 9 (Main Memory) and 10 (Virtual Memory).

[3] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed., Pearson, 2014. Chapter 3 (Memory Management).

[4] L. A. Bélády, "A study of replacement algorithms for a virtual-storage computer," *IBM Systems Journal*, vol. 5, no. 2, pp. 78–101, 1966.

[5] cppreference.com, C++ standard library reference (`std::optional`, `std::mt19937`, `std::deque`). `https://en.cppreference.com`